

Homework 4 Paper Questions: ResNets and Transformers

Due: Friday, November 21st at 11:59 pm

Deliverables. Submit a PDF of your write-up to Gradescope *HW4 Paper Questions*

Note: we **highly discourage** very long answers, most or all of your free response answers should be 1-3 sentences.

Overview

Machine learning architectures have evolved over time. Starting from the humble multi-layer perceptron (MLPs) to highly complex neural networks with hundreds of layers! In this homework, we'll explore 2 monumental architectural innovations: **ResNets** and **Transformers**.

You are encouraged to skim related works, but focus on the problems, current solutions, contributions, methods, and limitations.

This assignment is not about memorizing details, but about developing the ability to *read research papers methodically*. Most research papers follow a common structure: they first motivate a **problem**, then describe **current solutions** and their limitations, followed by the **proposed solution and key insights**, the **methods** used, and finally a discussion of **limitations**.

This homework is designed to help you practice and internalize this process of critical reading as you answer the questions. We have also provided pointers to relevant section that you might want to pay more attention to for each question.

- ResNets: Deep Residual Learning for Image Recognition (arXiv:1512.03385)
- Transformers: Attention Is All You Need (arXiv:1706.03762)

Some helpful resources compiled by course staff for understanding the Attention Is All You Need Paper:

- YouTube: Transformers, the tech behind LLMs — Deep Learning Chapter 5 - 3Blue1Brown
- YouTube: Attention in transformers, step-by-step — Deep Learning Chapter 6 - 3Blue1Brown
- YouTube: How might LLMs store facts — Deep Learning Chapter 7 - 3Blue1Brown
- YouTube: Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!! - StatQuest
- YouTube: Deep Dive into LLMs like ChatGPT
- The Transformer Architecture - Dive Into Deep Learning

1 ResNets

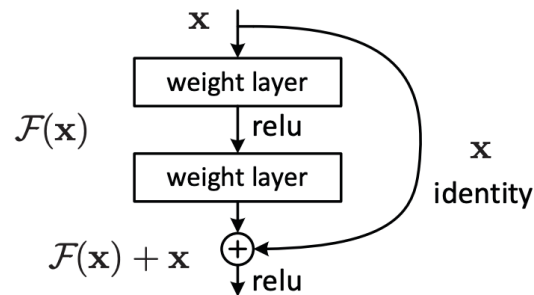


Figure 2 from the Deep Residual Learning for Image Recognition: "Residual learning: a building block."

Problem

Q1. (1 Pt.) In less than 5 words, describe the kind of tasks was ResNet designed for. [Sections: 1 - Introduction]

Solution. Image classification.

Q2. (1 Pt.) What problem did the authors of the ResNet paper aim to solve? [Sections: Abstract, 1 - Introduction]

Hint: What kind of problems did researchers/engineers notice happening with deeper neural networks? What differentiates **degradation** from **overfitting**?

Solution. Degradation: adding layers raised training error because optimization became harder. Overfitting instead concerns poor generalization despite fitting the training data.

Current works

Q3. (2 Pts.) How might you construct a deeper counterpart of a neural network that produces the exact same outputs as a shallower network? Why doesn't this work in practice, i.e. why does **degradation** occur in deeper neural networks? [Sections: 1 - Introduction, 2 - Related Work]

Solution. Copy the shallow network's parameters and insert identity layers. This preserves outputs, but ordinary training may fail to discover such a solution efficiently.

Proposed Solution

- Q4.** (1 Pt.) The ResNet authors proposed building layers that can learn the residual function $F(x) + x$ instead of directly learning $H(x)$. Why might we want to learn the function $F(x) + x$ if it is equivalent to $H(x)$? How does learning $F(x)$ help solve the degradation problem? [Section: 3.1 - Residual Learning]

Solution. Learn $F(x) = H(x) - x$, then output $F(x) + x$. Near identity, learning a small residual is easier than fitting identity with stacked nonlinear layers.

Method Details

- Q5.** (2 Pts.) What are two workarounds if your original input x and the output of your sub-layer $F(x)$ don't have equal dimensions? [Section: 3.3 - Network Architectures - Residual Network]

Solution. Use parameter-free zero-padding for extra channels, or a learned 1×1 projection. When spatial resolution decreases, downsample the shortcut with the corresponding stride.

- Q6.** (2 Pts.) Explain the 3 options for how residual/shortcut connections might be implemented. and compare their parameter counts (relative comparisons—which has more vs. less—are okay). Based on the parameter count comparisons, briefly discuss how the 3 different residual connection implementations might affect the model's memory requirements or training time. [Section: 4 - Experiments - Identity vs. Projection Shortcuts]

Solution. (A) Identity shortcuts with zero-padding add no learned parameters; (B) projections only at dimension changes add some; (C) projections everywhere add the most. For the same backbone, parameter storage and projection computation generally increase from A to B to C, increasing training memory and time.

Key Insight & Contributions

- Q7.** (1 Pt.) What benefits did the authors notice with residual networks? What empirical proof did the authors provide to show that ResNet architectures didn't suffer from the degradation problem? [Sections: Abstract, 4 - Experiments, Figure 4, Table 2]

Solution. Residual networks converge more easily and benefit from depth. Figure 4 shows lower training error for ResNet-34 than ResNet-18; Table 2 shows validation error falling from 27.88% to 25.03%, while deeper plain networks worsen.

2 Transformers - Attention Is All You Need

A helpful preface:

1. Transduction (AKA seq2seq) refers to the task of mapping input sequences to output sequences. For example, machine translation refers to a model translating a *source sequence* in one language into a *target sequence* in another language.
2. During generation, the decoder predicts the token at position i . The input embedding the decoder receives comes from the first $i - 1$ output tokens. This is called **teacher forcing** during training (feeding in ground-truth tokens shifted by one) and **auto-regressive generation** at inference. Offsetting by one during training ensures the model's prediction for position i depends on previous outputs, never on itself or future targets.

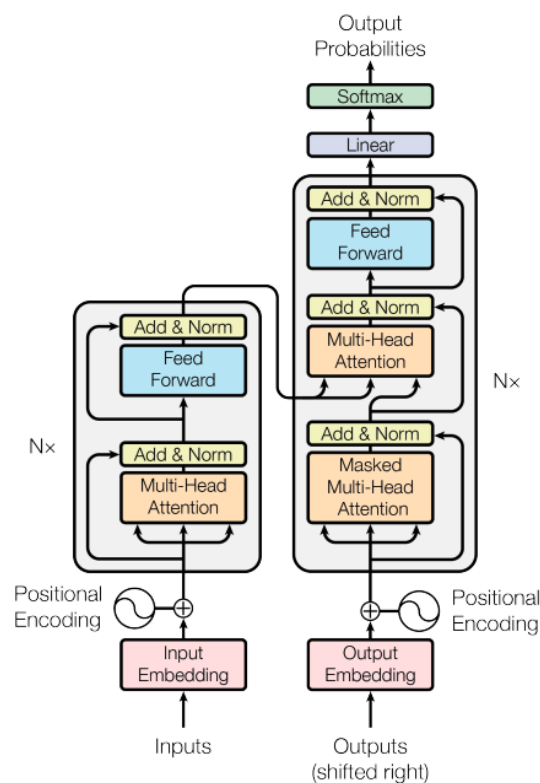


Figure 1 from Attention Is All You Need: "The Transformer - model architecture."

Problem

Q8. (1 Pt.) In a short sentence, What task is the paper "Attention Is All You Need" trying to solve? *[Section: 1- Introduction]*

Solution. The paper addresses sequence-to-sequence modeling, especially machine translation.

Q9. (1 Pt.) What are the jobs of the encoder and decoder? *[Section: 3 - Model Architecture]*

Solution. The encoder maps the source sequence to contextual representations. The decoder uses those representations and the preceding output tokens to predict the next token.

Metrics

Q10. (2 Pts.) Two important metrics that the authors highlight in the paper are BLEU scores and perplexity. Describe what BLEU score and perplexity measure, and what their limitations are. Feel free to use some outside research! *[Sections: Abstract, Table 3]*

Solution. BLEU measures clipped n -gram agreement with reference translations, with a brevity penalty; higher is better, but valid paraphrases can score poorly and word overlap does not reliably establish meaning or correctness. Perplexity is $\exp[-N^{-1} \sum_{t=1}^N \log p(y_t | y_{<t}, x)]$, so lower values indicate higher probability assigned to the reference tokens. It depends on tokenization and evaluation data, making unmatched comparisons misleading, and does not directly measure generated-text quality.¹

Current Works

Q11. (1 Pt.) What are some challenges that existing architectures face? Describe the shortcomings of both recurrent neural networks (RNNs) and sequential convolutional neural networks. *[Sections: 1 - Introduction, 2 - Background]*

Solution. RNN states depend on preceding states, restricting parallelism within a sequence and slowing long-sequence training. Convolutional sequence models process positions in parallel, but linking distant positions requires multiple layers, making long-range dependencies harder to learn.

Proposed solution

Q12. (1 Pt.) In a short sentence, describe what "attention" is in your own words. This doesn't have to be within the context of machine learning, but feel free to start also thinking about how the idea of attention might transfer over to what the transformer architecture tries to do. *[Section: your mind]*

Solution. Attention means giving more weight to information that is relevant to the current task.

¹BLEU: Papineni et al. (2002).

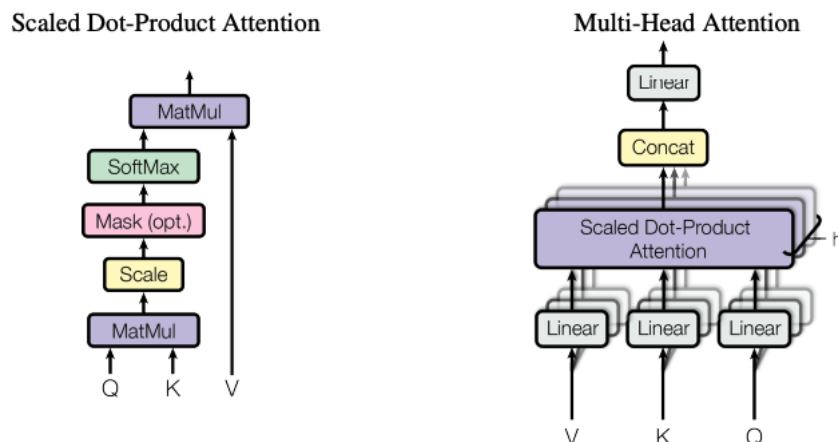


Figure 2 from Attention Is All You Need: "(left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel."

2.1 Method Details

Q13. (1 Pt.) What is the purpose of masking during decoder self-attention? [Sections: 3.1 - Encoder and Decoder Stacks, 3.2.3 - Applications of Attention in our Model]

Solution. A causal mask blocks attention to future positions, so each prediction uses only the permitted preceding target tokens. Together with shifted decoder inputs, it prevents target leakage during parallel training.

Q14. (2 Pts.) Why does the Transformer paper scale QK^T by $\frac{1}{\sqrt{d_k}}$? [Section: 3.2.1 - Scaled Dot-Product Attention]

Hint: Imagine we sum up I.I.D. random variables where $x_i \sim \mathcal{N}(0, 1)$. What is the variance of this sum?

Solution. For mutually independent, zero-mean, unit-variance components, each product $q_i k_i$ has variance 1, hence

$$\text{Var}(q^\top k) = d_k, \quad \text{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) = 1.$$

The scaling keeps attention scores from growing with dimension, reducing softmax saturation and the resulting small gradients.

Q15. (1 Pt.) What are 2 benefits of multi-headed attention? [Section: 3.2.2 - Multi-Head Attention]

Solution. Different heads can attend to different positions and learned representation subspaces simultaneously. Keeping their weighted summaries separate until concatenation preserves distinctions that a single weighted average can mix together.

Q16. (2 Pts.) Where do the query, key, and value vectors come from in the following parts of a transformer, and why? [Section: 3.2.3 - Applications of Attention in our Model]

- (a) Encoder self-attention
- (b) Decoder self-attention
- (c) Decoder cross-attention (the paper refers to this as "encoder-decoder attention")

Solution. In encoder self-attention, Q, K, V are separate learned projections of the current encoder layer's input, allowing source positions to exchange information. In decoder self-attention, all three come from the current decoder layer's input, with a causal mask restricting target-context access. In cross-attention, Q comes from the preceding decoder sublayer and K, V from the encoder's final output, so the decoder can retrieve source information relevant to its next prediction.

Q17. (2 Pts.) Explain the differences between what the transformer decoder receives as input and what it outputs during training vs. inference. [Sections: 3 - Model Architecture, 5.1 - Training Data and Batching]

Solution. During training, the decoder receives the ground-truth target sequence shifted right and predicts next-token distributions at all positions in parallel under a causal mask (teacher forcing). During inference, it begins with a start token, receives its own generated prefix, and selects one new token per step until an end token or length limit. In both cases it also attends to the encoder's source representations.

Q18. (2 Pts.) Why does the standard input to a transformer model (before adding positional encodings) not contain any information about the order of tokens in a sequence? How do positional encodings help the model understand token order? [Section: 3.5 - Positional Encoding]

Solution. Token embeddings encode identity rather than position, and unmasked self-attention alone is permutation-equivariant: rearranging inputs rearranges outputs without supplying an order signal. Adding a position-dependent vector to each embedding lets attention use absolute positions and learn relative positional relationships.

Key Insight / contributions

Q19. (2 Pts.) What advantages does the self-attention mechanism have over recurrent layers and convolutional layers in the context of sequence modeling? [Section: 4 - Why Self-Attention]

Solution. Self-attention processes positions in parallel, whereas recurrent layers require sequential steps. It connects any two positions within one layer, shortening information paths compared with recurrence and local convolutions and helping model long-range dependencies. Its per-layer cost $O(n^2d)$ is lower than a recurrent layer's $O(nd^2)$ when $n < d$, though quadratic dependence on sequence length makes long inputs expensive.

Future Work

Q20. (2 Pts.) As mentioned earlier in the assignment, the transformer architecture has rocked the machine learning world in countless ways! One application has been in computer vision, where Vision Transformers have matched and even beat state-of-the-art convolutional neural networks in image-processing tasks. How might you adapt transformers to process images? Think about how you might create "tokens" from images. If you don't know where to start, feel free to skim follow-up works to this paper which look into this.

Solution. Divide the image into fixed-size patches, flatten each patch, and linearly project it to a token embedding; for example, a 224×224 image gives 196 patches of size 16×16 . Add positional embeddings and process the patch sequence with a Transformer encoder. A learned classification token can collect image information through attention, and its final representation feeds a classification head.²

Q21. (2 Pts.) Suppose you have a ResNet and a Vision Transformer, both trained on ImageNet with standard preprocessing (e.g. image resizing, pixel standardization). Now, you are given a new image of size 1024×512 . How would you process this image for each model? What are the key differences in how ResNets and Vision Transformers handle input image sizes?

Solution. For either model, first use the checkpoint's expected preprocessing, such as aspect-preserving resizing and a 224×224 center crop followed by its channel normalization. To retain 1024×512 , a standard ResNet's convolutions and adaptive global pooling can accommodate the changed feature-map size, while a standard ViT needs dimensions compatible with its patch size and interpolation of its learned patch-position grid. With 16×16 patches, the grid grows from 14×14 to 64×32 (196 to 2048 tokens, excluding any classification token), substantially increasing attention cost; the classification-token position is kept separate, and accuracy at the new resolution is not guaranteed.

Acknowledgement

OpenAI Codex assisted with explanations, drafting, and checking the answers against the cited papers.

²Vision Transformer: Dosovitskiy et al., An Image is Worth 16x16 Words.